

# LABORATORIO DE TECNOLOGÍA: CONTENERIZACIÓN Y DESPLIEGUE

Guía de lectura y configuración de entornos con Docker

Por: Matías Barreto • basado en la clase de Kevin Barroso

Docker

WSL 2

IA y datos

Principiantes +  
profesionales

## 1 ¿Por qué Docker?



- ✓ Empaqueta código + dependencias
- ✓ Portable, reproducible y escalable
- ✓ Evita: "en mi máquina sí funcionaba"
- ★ Ideal para IA, notebooks y flujos de datos

## 2 VM vs Contenedor



Máquina Virtual



Contenedor

|                 |              |                 |
|-----------------|--------------|-----------------|
| Arquitectura    | SO invitado  | comparte kernel |
| Peso y arranque | GB y minutos | MB y segundos   |
| Aislamiento     | total        | por procesos    |

## 3 Los 3 pilares de Docker



Dockerfile  
receta



Imagen  
molde congelado



Contenedor  
app en ejecución

## 4 Proyecto reproducible



app.py /  
notebook.ipynb



requirements.txt



Dockerfile



docker-  
compose.yml



.dockerignore

✓ Todo lo necesario para reconstruir el entorno

## 5 Instalación rápida

Windows

- 1 Virtualización habilitada
- 2 wsl --install / wsl --update
- 3 Reiniciar
- 4 Docker Desktop + WSL 2

macOS

- 1 Elegir Apple Silicon o Intel
- 2 Rosetta 2 (opcional)
- 3 Abrir Docker Desktop

## 6 Troubleshooting + práctica

A. Problemas frecuentes

- "docker" no reconocido → abrir Docker Desktop / reiniciar VS Code
- WSL o Engine detenido → WSL Integration + wsl --update
- Jupyter no abre → ports: "8888:8888"

## B. Actividad sugerida

- 1 Clonar repositorio
- 2 Abrir en VS Code
- 3 docker - compose up
- 4 Entrar a http://localhost:8888
- 5 Ejecutar celdas

Glosario rápido:



Host  
máquina que ejecuta  
los contenedores



Kernel  
núcleo del SO que  
gestiona recursos



Puerto  
puerta de entrada  
de un servicio



Variable de entorno  
configuración clave  
para la aplicación



Volumen  
almacenamiento  
persistente



**Idea clave:** Docker estandariza el entorno para que el software funcione igual en cualquier máquina.

le

# LABORATORIO DE TECNOLOGÍA: CONTENERIZACIÓN Y DESPLIEGUE

Guía de Lectura y Configuración de Entornos con Docker

Por: Matías Barreto (basado en la clase de Kevin Barroso)

## Introducción y Contexto

En el desarrollo de modelos de Inteligencia Artificial (NLP, Visión Computacional) y flujos de datos, el código es solo una parte de la ecuación. El verdadero desafío logístico surge al llevar ese modelo a un entorno productivo. Cuando trabajamos con Notebooks o scripts en Python, el reto es lograr que ese proceso sea escalable, seguro y, sobre todo, que **no se rompa al cambiar de computadora**.

Piensa en el transporte marítimo del siglo pasado: se cargaban barcos con cajas, barriles y bolsas de distintos tamaños; era un caos logístico. La solución que revolucionó el mundo fue **el contenedor de envío estándar**. No importa si transporta autos o manzanas, el contenedor tiene el mismo tamaño y los barcos saben exactamente cómo apilarlos.

En ingeniería de software, Docker hace exactamente lo mismo. Un contenedor empaqueta tu código con todas sus dependencias (como *Pandas*, *Scikit-Learn* o *Transformers*). Esto elimina para siempre la temida frase: "*Pero en mi máquina sí funcionaba*".

## I. Virtualización vs. Contenerización

Comprender esta diferencia es vital para entender por qué usamos Docker:

| Característica   | Máquina Virtual (VM)                                                        | Contenedor (Docker)                                                     |
|------------------|-----------------------------------------------------------------------------|-------------------------------------------------------------------------|
| Arquitectura     | Simula hardware completo y requiere su propio Sistema Operativo "invitado". | Comparte el núcleo (Kernel) del Sistema Operativo de tu máquina física. |
| Peso y Velocidad | Muy pesadas (Gigabytes). Tardan minutos en arrancar.                        | Ligeros (Megabytes). Arrancan en segundos.                              |
| Aislamiento      | Total (Hardware y Software).                                                | A nivel de procesos (Software).                                         |

## Los Tres Pilares de Docker

- **Dockerfile:** El *recetario*. Es un archivo de texto con las instrucciones paso a paso (qué sistema operativo base usar, qué librerías instalar, qué archivos copiar).
- **Imagen:** El *molde congelado*. Es el archivo estático y empaquetado que resulta de construir el [Dockerfile](#).
- **Contenedor:** La *aplicación viva*. Es la imagen en ejecución. Puedes tener múltiples contenedores idénticos corriendo a partir de una sola imagen.

## II. Anatomía de un Proyecto Contenerizado

Para que un proyecto sea 100% reproducible, tu repositorio debe verse similar a esto:

-  `app.py` o `notebook.ipynb` (Tu código fuente).
-  `requirements.txt` (Versiones exactas de tus librerías, ej: `pandas==2.1.0`).
-  `Dockerfile` (Las instrucciones de construcción de la imagen).
-  `docker-compose.yml` (Archivo que orquesta y configura los servicios, como mapear el puerto 8888 del contenedor al 8888 de tu PC física).
-  `.dockerignore` (Funciona como un `.gitignore`. Evita que archivos pesados o innecesarios, como la carpeta `.venv` o datos temporales, entren a la imagen).

## III. Guía de Instalación (Cero Frustración)



### Windows (El escenario más común y complejo)

Docker en Windows necesita un subsistema de Linux para funcionar correctamente. **Sigue estos pasos en orden estricto:**

#### Paso 1: Habilitar la Virtualización

- Debes asegurarte de tener habilitada la virtualización de hardware en la BIOS/UEFI de tu computadora. (Para comprobarlo: Abre el Administrador de Tareas > pestaña Rendimiento > CPU > busca "Virtualización: Habilitado").

- Tu sistema debe ser Windows 10/11 de 64 bits.

### **Paso 2: Instalar WSL2 (Windows Subsystem for Linux)**

1. Abre **PowerShell** como Administrador (clic derecho > Ejecutar como administrador).
2. Escribe el comando: `wsl --install`.
3. Si ya lo tenías, asegúrate de actualizarlo con: `wsl --update`.
4. **REINICIA TU COMPUTADORA**. Este paso es innegociable.

### **Paso 3: Instalar Docker Desktop**

1. Descarga Docker Desktop para Windows.
2. Ejecuta el instalador. Se recomienda la instalación "por usuario" (per-user) para evitar conflictos de permisos.
3. Durante la instalación, asegúrate de que la opción **"Use WSL 2 instead of Hyper-V"** esté marcada.
4. Al finalizar, abre Docker Desktop y acepta los términos. Deja la aplicación abierta en segundo plano.

## **macOS**

1. Verifica tu procesador (Menú Apple > Acerca de esta Mac). Descarga la versión correspondiente: **Apple Silicon** (M1/M2/M3) o **Intel**.
2. *Nota para Apple Silicon:* Se recomienda instalar Rosetta 2 desde la terminal (`softwareupdate --install-rosetta`) para máxima compatibilidad.
3. Abre la aplicación para conceder los permisos iniciales.

## **IV. Solución de Problemas Frecuentes**

**"Abro la terminal de VS Code, escribo `docker` y me dice: comando no reconocido."**

- **Causa 1:** Docker Desktop no está abierto. Busca el ícono de la ballena en la barra de tareas.
- **Causa 2:** Debes reiniciar VS Code por completo para que reconozca las nuevas rutas (PATH) del sistema.

**"Me da un error relacionado a WSL o dice que el motor (Engine) está detenido."**

- **Solución:** En Docker Desktop, ve a **Settings > Resources > WSL Integration**. Asegúrate de que la integración esté habilitada con tu distribución por defecto (usualmente Ubuntu). Luego ejecuta `wsl --update` en PowerShell.

**"Mi contenedor levanta, pero no puedo acceder a Jupyter Lab en el navegador."**

- **Solución:** Revisa tu archivo `docker-compose.yml`. Si Jupyter corre internamente en el puerto 8888, tu configuración de puertos debe ser obligatoriamente: `ports: - "8888:8888"`.

## V. Actividad Práctica Sugerida

1. Descarga el repositorio de GitHub proporcionado en clase.
2. Verifica que Docker Desktop esté abierto (ballena en verde).
3. Abre la carpeta del proyecto en VS Code.
4. Abre la terminal integrada y ejecuta: `docker-compose up`.
5. Docker descargará la imagen base (ej. Debian Slim), instalará las dependencias y levantará el servidor.
6. Accede a la URL de Jupyter Lab que aparecerá en la terminal (ej: <http://localhost:8888>).
7. Ejecuta las celdas y observa cómo tu modelo de IA corre perfectamente sin haber instalado Python ni librerías en tu sistema local.



## Glosario Rápido para Nivelación

- **Host:** Tu computadora física (donde corre Docker).
- **Kernel:** El corazón del sistema operativo que gestiona el hardware.
- **Puerto:** Una "puerta" virtual de comunicación (ej. el 80 para web, 8888 para Jupyter).
- **Variable de Entorno:** Un valor dinámico (como una contraseña o una ruta) que el software lee para saber cómo configurarse.
- **Volumen:** Una carpeta de tu computadora conectada al contenedor para que los datos no se borren al apagarlo.